

Testes Unitários em Python

```
$ pytest test_soma.py
```

```
1 passed in 0.01s
```

unittest vs pytest

Prática 01 · Seminário — Comparação de bibliotecas de testes unitários em uma mesma linguagem

Apresentado por: **Hadriel Correia Gomes** (RA 326127495) · **Júlia de Souza Aguiar** (RA 32328729)

O que são testes unitários?

Prática que verifica, de forma automatizada, se cada componente do código (função, método, classe) se comporta como esperado — isoladamente dos demais.

- Detecta erros cedo, antes da integração
- Documenta o comportamento esperado do código
- Dá segurança para refatorar sem quebrar nada

Bibliotecas por linguagem

Python	unittest, Pytest
Java	JUnit, TestNG
JavaScript	Mocha, Jest
.NET (C#)	xUnit, NUnit
Web / E2E	Cypress, Playwright

Foco desta apresentação: duas bibliotecas de Python — unittest e Pytest.



Biblioteca nativa da linguagem Python (módulo da standard library), inspirada no JUnit e baseada em classes.

Pontos positivos

- + Já vem com o Python — não precisa instalar nada
- + Estrutura orientada a objetos, familiar a quem já usou JUnit/NUnit
- + API madura, estável e amplamente documentada
- + Boa integração com ferramentas de CI antigas

Pontos negativos

- Sintaxe mais verbosa (classes, métodos assertXxx)
- Fixtures limitadas: apenas setUp/tearDown
- Mensagens de falha menos detalhadas
- Parametrização de testes não é nativa

Exemplo de uso — unittest

```
test_soma.py

import unittest
from operacoes import soma

class TestSoma(unittest.TestCase):

    def test_soma_positivos(self):
        self.assertEqual(
            soma(2, 3), 5
        )

    def test_soma_negativos(self):
        self.assertEqual(
            soma(-1, -1), -2
        )

if __name__ == '__main__':
    unittest.main()
```

saída no terminal

```
$ python -m unittest test_soma.py
..
-----
Ran 2 tests in 0.001s

OK
```

diferenças de implementação

- Exige classe herdando de TestCase
- Comparações via métodos: assertEquals, assertTrue...
- Execução: python -m unittest
- Saída resumida (pontos e resumo final)



Framework de terceiros, instalado via pip, com sintaxe enxuta baseada em funções e no assert nativo do Python.

Pontos positivos

- + Sintaxe simples: funções com assert comum do Python
- + Fixtures poderosas e reutilizáveis (`@pytest.fixture`)
- + Parametrização nativa de testes (`@pytest.mark.parametrize`)
- + Mensagens de falha detalhadas e grande ecossistema de plugins

Pontos negativos

- Não vem embutido — precisa instalar (`pip install pytest`)
- Reescreve o assert internamente ('magic'), o que pode confundir iniciantes
- Menos rígido estruturalmente, exigindo disciplina do time
- Curva de aprendizado extra para fixtures avançadas

Exemplo de uso — pytest

```
test_soma.py

from operacoes import soma

def test_soma_positivos():
    assert soma(2, 3) == 5

def test_soma_negativos():
    assert soma(-1, -1) == -2

# parametrização nativa
import pytest

@pytest.mark.parametrize(
    "a,b,esperado",
    [(2, 3, 5), (-1, -1, -2)]
)
def test_soma_parametrizado(a, b, esperado):
    assert soma(a, b) == esperado
```

saída no terminal

```
$ pytest test_soma.py -v
test_soma_positivos PASSED
test_soma_negativos PASSED
test_soma_parametrizado[2-3-5] PASSED
test_soma_parametrizado[-1--1--2] PASSED

==== 4 passed in 0.02s ====
```

diferenças de implementação

- Não precisa de classe: funções soltas
- assert nativo do Python, sem métodos especiais
- Execução: pytest (descoberta automática)
- Saída verbosa e legível, com diffs detalhados em falhas

Comparação direta

Critério	unittest	pytest
Origem	Standard library (nativo)	Pacote externo (pip install)
Estrutura	Baseada em classes (TestCase)	Baseada em funções simples
Sintaxe de verificação	<code>self.assertEqual(a, b)</code>	<code>assert a == b</code>
Execução	<code>python -m unittest</code>	<code>pytest</code>
Parametrização	Manual / repetição de métodos	<code>@pytest.mark.parametrize</code> nativo
Saída em caso de falha	Mensagem básica	Diff detalhado do valor esperado x obtido

Conclusão

Use unittest quando...

- Quer evitar dependências externas
- O time já conhece bem estrutura JUnit-like
- O projeto é pequeno ou legado, sem pytest instalado

Use pytest quando...

- Quer código de teste mais limpo e produtivo
- Precisa de fixtures e parametrização avançadas
- O projeto é novo e pode instalar dependências

Na prática, muitos projetos usam pytest como executor mesmo para testes escritos com unittest — as duas abordagens são compatíveis.